Lab Report

on

Software Engineering and Design, (BCSE0653)

Session: 2025-26 (Even), Batch: 2023-27

Department of CS

Noida Institute of Engineering and Technology, Greater Noida

Submitted by

Student Name:

Amit Kumar

(ERP: 0231CS075, Roll No.: 2301330120020)

Submitted to

Ms. Arhina Ghosh

(Assistant Professor)

Department of CS

April, 2026

Course Objectives:

To practically implement Software Engineering concepts through the development of Learnify, a real-time web-based learning system, focusing on low-latency communication, user engagement, secure authentication, system scalability, performance evaluation, and learning analytics.

Course Outcome:

CO	**Course Outcome:** After completion of the course, the student will be able to	**Bloom's Levels**
CO1	**Analyze system requirements** and understand the design of a real-time online learning platform.	K4
CO2	**Apply SDLC phases** including analysis, design, development, testing, and maintenance.	K3
CO3	**Design and implement real-time features** such as video conferencing, chat, and screen sharing.	K4
CO4	**Apply software testing techniques** to ensure functionality and performance.	K3
CO5	**Implement security practices** including secure authentication and data protection.	K3

1. Mode

Hands-on + Concept Explanation

- Offline / Lab-based Implementation
- 70% Practical, 30% Theory
- Real-time feature demonstrations
- Lab-based development and testing sessions

2. Target Audience

- Students

3. Objectives

The main objectives of the **Learnify** project are:

- **To design and develop a real-time online learning web application** that enables low-latency video and audio communication using modern web technologies.
- **To implement interactive learning features** such as live chat, emoji reactions, and screen sharing to enhance student engagement.
- **To apply secure authentication and data protection mechanisms** ensuring safe user access and encrypted communication.

- **To build a scalable and lightweight system architecture** using the MERN stack suitable for multi-user real-time communication.
- **To collect and analyze learning engagement metrics** such as attendance, chat activity, and reactions for performance and research evaluation.

4. Prerequisites

Basic knowledge of computer fundamentals and software development processes.

5. Tools / Software Required

- React.js
- Express.js
- MongoDB

6. Methodology

▪ Requirements were gathered and analyzed for a real-time online learning system.
▪ System architecture and user interface were designed using MERN principles.
▪ Frontend and backend modules were developed and integrated.
▪ Real-time communication features were implemented using WebRTC and Socket.IO.
▪ The system was tested for functionality, performance, and reliability.

Proposed List of Practical for Workshop

S.NO.	PRACTICAL (Suggestive List of Practical)	Date	Page No
1.	Team formation and allotment of Mini project: Problem statement, Literature survey, Requirement. analysis.		
2.	Draw the use case diagram		
3.	Draw the Data Flow Diagram (DFD): All levels.		
4.	Design an ER diagram for with multiplicity.		
5.	Prepare SRS document in line with the IEEE recommended standards.		
6.	Draw State chart diagram.		
7.	Draw Object and Class diagram.		
8.	Create Interaction diagram: sequence diagram for SDD.		
9.	Create Interaction diagram: collaboration diagram for SDD.		
10.	Create Activity diagram.		
11.	Create Component diagram.		
12.	Create Deployment diagram.		
13.	Estimation of Test Coverage Metrics and Structural Complexity.		
14.	Design and develop a program in a language of your choice to solve the triangle problem defined as follows: Accept three integers which are supposed to be the three sides of a triangle and determine if the three values represent an equilateral triangle, isosceles triangle, scalene triangle, or they do not form a triangle at all. Assume that the upper limit for the size of any side is 10. Derive test cases for your program based on boundary-value analysis, execute the test cases, and discuss the results.		
15.	Design, develop, code, and run the program in any suitable language to solve the commission problem. Analyze it from the perspective of boundary value testing, derive different test cases, execute these test cases, and discuss the test results.		
16.	Design and develop a program in a language of your choice to solve the triangle problem defined as follows: Accept three integers which are supposed to be the three sides of a triangle and determine if the three values represent an equilateral triangle, isosceles triangle, scalene triangle, or they do not form a triangle at all. Assume that the upper limit for the size of any side is 10.		

	Derive test cases for your program based on equivalence class partitioning, execute the test cases, and discuss the results.		

17.	Design and develop a program in a language of your choice to solve the triangle problem defined as follows: Accept three integers which are supposed to be the three sides of a triangle and determine if the three values represent an equilateral triangle, isosceles triangle, scalene triangle, or they do not form a triangle at all. Derive test cases for your program based on decision-table approach, execute the test cases, and discuss the results.		
18.	Create test cases for a program which determine whether an integer is prime or not by using path testing.		
19.	Create test cases for a program which determine whether an integer is prime or not by using Cyclomatic complexity.		
20.	Consider a program to input two numbers and print them in ascending order. Find all du paths and identify those du-paths that are not feasible. Also find all dc paths and generate the test cases for all paths (dc paths and non dc paths).		
21.	Consider the code to arrange the nos. in ascending order. Generate the test cases for loop coverage and path testing. Check the adequacy of the test cases through mutation testing and compute the mutation score for each.		
22.	Write Test cases for any Known Application (e.g., Banking Application)		
23.	Create a test plan document for any application (e.g., Library Management System)		
24.	Study of any testing tool (e.g., Win Runner)		
25.	Study of any bug tracking tool (e.g., Bugzilla, Bug bit)		
26.	Study of any test management tool (e.g., Test Director)		
27.	Study of any open source-Testing tool (e.g., Test link, Test Rail)		
28.	Study of any web testing tool (e.g., Selenium).		
29.	Mini Project with CASE tools.		
30.	Case Study Provided by Industry.		

PRACTICAL NO-1

Objective: Design and development of a Real-Time Online Learning Web Application (Learnify) integrating live video conferencing, chat interaction, secure authentication, and engagement analytics.

Work Done / Result:

1. **Team Formation**
   A project team was formed with defined roles to ensure efficient development and coordination:
   • Frontend Development – UI design, video interface, chat system
   • Backend Development – APIs, authentication, signaling server
   • Database Management – MongoDB schema & storage

2. **Problem Statement**
   Existing online learning platforms face multiple challenges:
   • High latency in video/audio communication
   • Limited student engagement tools
   • Lack of integrated analytics
   • Heavy and complex system architectures
   • Weak real-time interaction mechanisms
   To address these issues, **Learnify** was proposed as a lightweight, browser-based real-time learning system combining communication, interaction, and analytics.

3. Literature Survey
Various existing solutions were studied:
• Traditional video conferencing tools
• LMS platforms
• Chat-based classroom systems

Limitations Identified:

• Poor real-time interaction
• Lack of engagement measurement
• High bandwidth consumption
• Limited customization for education
• Weak integration of communication + analytics

This analysis highlighted the need for a unified, scalable real-time learning platform.

3. **Requirement Analysis**
   Functional Requirements
   • The system must support user registration and secure login.
   • It should enable real-time video/audio conferencing and screen sharing.
   • A live chat and reaction emoji feature must be provided.
   • The platform should manage rooms, attendance, and engagement analytics.

Non-Functional Requirements

• The system must ensure secure authentication and encrypted data handling.
• It should maintain low latency and high-quality media streaming.
• The architecture must be scalable, reliable, and responsive.
• The interface should be user-friendly and compatible across browsers.

S.No	Author / Year	Title	Research Objective	Result	Strength	Limitation
1	Anderson et al. (2020)	*A Review of WebRTC-Based Real-Time Communication Systems*	To analyze WebRTC performance in browser-based real-time communication	Demonstrated low-latency peer-to-peer media exchange	Validates WebRTC effectiveness for real-time communication	Not focused on education platforms
2	Pappas et al. (2019)	*Enhancing Student Engagement in Online Learning Environments*	To study methods for improving student interaction	Chat and reactions increased engagement	Supports integration of engagement tools	Lacks real-time communication implementation
3	Khandelwal & Gupta (2021)	*Secure Authentication Mechanisms in Web Applications*	To evaluate modern web security techniques	bcrypt and encryption improved login security	Justifies secure authentication design	Focused on general web applications, not e-learning
4	Chen et al. (2018)	*Real-Time Video Streaming Over Web-Based Platforms*	To study efficient video streaming techniques	Reduced delay using optimized pipelines	Relevant to video streaming module	Not a complete educational platform
5	Martin & Parker (2022)	*Learning Analytics in Digital Education Systems*	To analyze analytics usage in education	Identified engagement and attendance metrics	Supports analytics integration	No real-time communication integration
6	Alshamrani (2019)	*Cloud-Based E-Learning Systems: Benefits & Challenges*	To study scalability in e-learning systems	Highlighted scalability and accessibility benefits	Relevant for system scaling and deployment	Lacks focus on WebRTC technology
7	O'Connor et al. (2023)	*Real-Time Collaborati*	To evaluate impact of	Improved communication efficiency	Supports live	Limited technical

		on Tools in Education	collaboration tools		interaction features	implementation details
8	Patel & Shah (2020)	*Performance Evaluation of Real-Time Communication Apps*	To measure RTC latency and throughput	WebRTC apps showed better performance	Validates WebRTC-based design	Not classroom-specific
9	IEEE Study (2021)	*Screen Sharing and Media APIs in Web Applications*	To analyze display media streaming	getDisplayMedia API proved effective	Supports screen sharing feature	Narrow scope limited to display media
10	Gomez et al. (2024)	*Lightweight MERN Stack Applications for Scalable Systems*	To evaluate MERN stack efficiency	MERN confirmed scalable and responsive	Supports MERN-based architecture	General application study, not education-specific

PRACTICAL NO-2

Objective: Draw the use case diagram.

Work Done / Result:

Definition:
A Use Case Diagram is a UML (Unified Modeling Language) diagram that represents how different actors (users or external entities) interact with the system through various functionalities (use cases). It provides a high-level functional overview of the system without detailing internal implementation.

Purpose of Use Case Diagram

- To understand system functionality from the user's perspective
- To identify system users (actors) and their roles
- To visualize interactions between actors and the system
- To support requirement analysis and system design

Main Components

Actors

Actors represent external entities interacting with Learnify.

Actors Identified:

- User
- Admin

Use Cases

Use cases represent system services or actions**.**

Major Use Cases in Learnify:

User:

- Register
- Login
- Create Meeting Room
- Join Meeting Room
- Leave Meeting
- Start Video
- Stop Video
- Mute/Unmute Audio
- Share Screen
- Send Chat Message
- Receive Chat Message
- End Call

Admin:

- Manage Rooms
- Monitor Active Sessions

• View Users & Sessions
• Manage Meetings

System Boundary

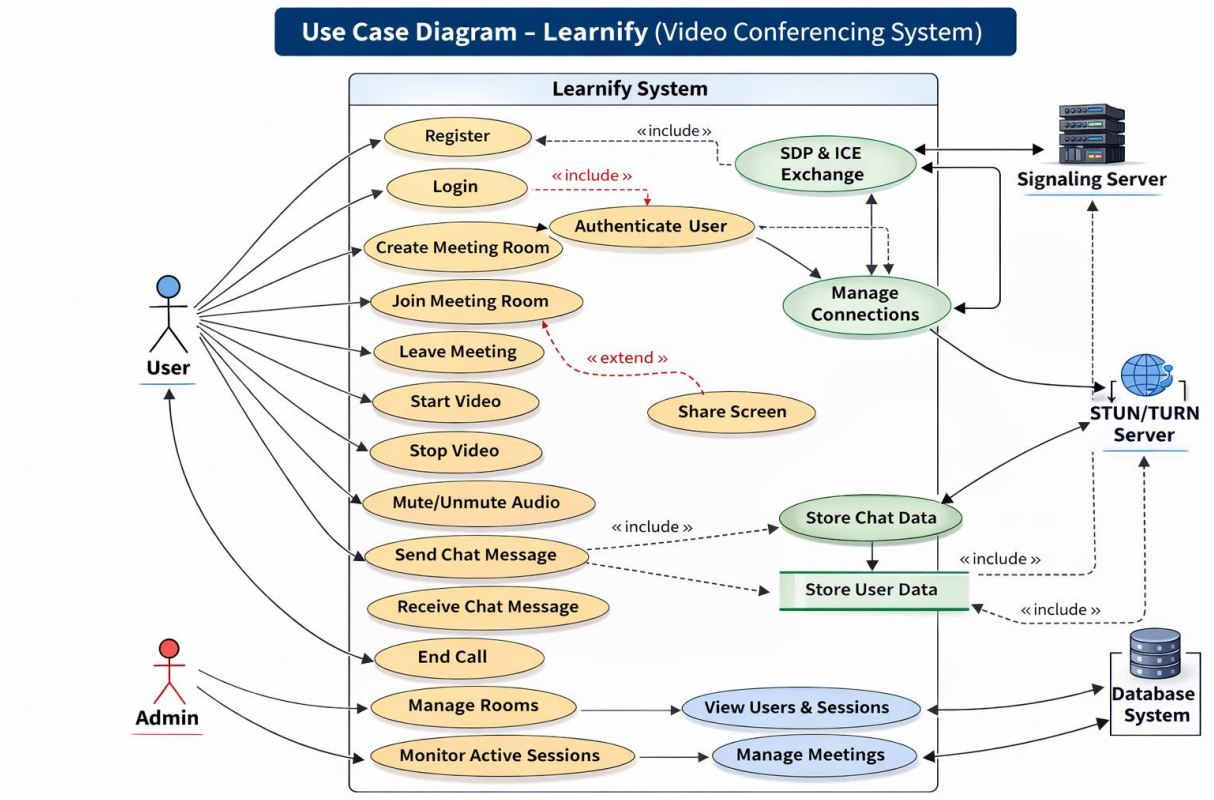
The system boundary defines the scope of the **Learnify System**.
All use cases are enclosed within this boundary.

Relationships

Association

Shows direct interaction between actors and use cases.

Examples:
• User → Login
• User → Join Meeting Room
• Admin → Manage Rooms



PRACTICAL NO-3

Objective: Draw the Data Flow Diagram (DFD): All levels.

Work Done / Result:
Definition:
A Data Flow Diagram (DFD) is a graphical representation that illustrates how data moves within a system, how it is processed, and where it is stored.
It focuses on the logical flow of information rather than technical implementation details.

Purpose of DFD
• To understand how data enters, flows through, and exits the system
• To visualize processes and data interactions
• To assist in system analysis and design
• To identify missing or redundant data flows Main Components of DFD

Process (Circle / Rounded Rectangle)

Represents activities that transform input data into output.

Examples:
- User Authentication
- Video Conferencing
- Chat Processing
- Screen Sharing
- Connection Signaling

Data Flow (Arrow)

Represents movement of data within the system.

Examples:
• User Auth & Info
• Video & Audio Data
• Chat Data
• Connection Signals

Data Store (Parallel Lines)

Represents storage of persistent system data.

Examples:
• User Database
• Chat Logs
• Session / Meeting Records
• Engagement Metrics

External Entity (Rectangle)

Represents actors interacting with the system.

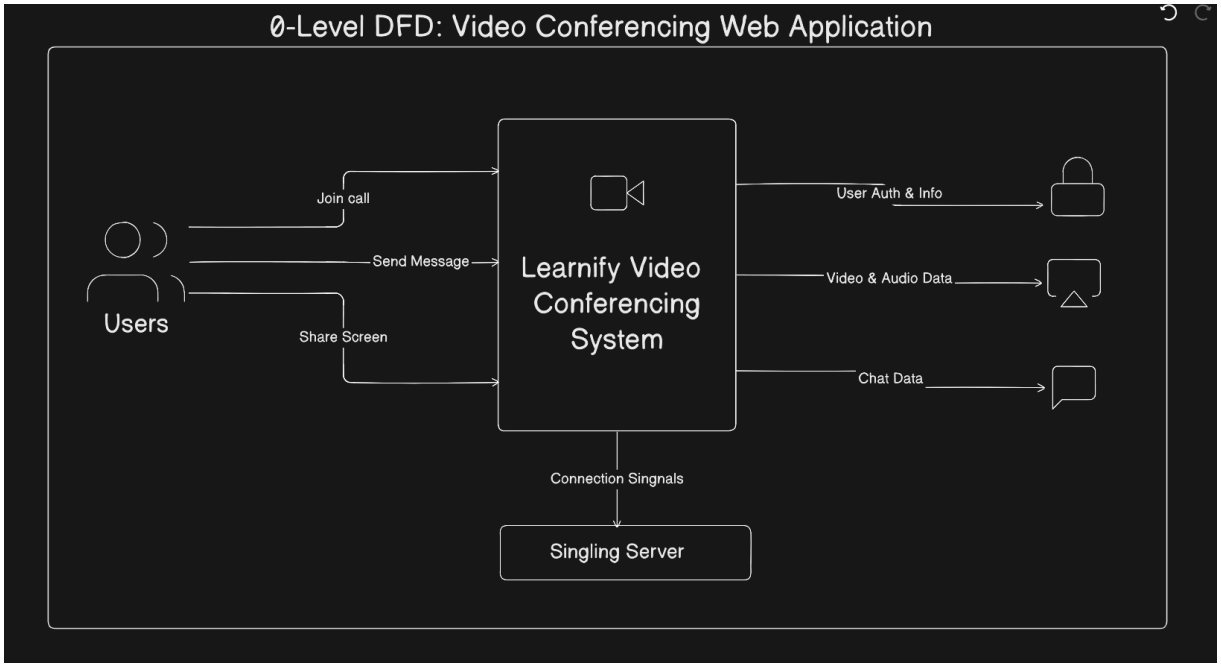
Entities:
• Users
• Admin

Levels of DFD

Level 0 – Context Diagram

• Shows Learnify as a single process
• Displays interaction with external entities
• No internal details

For Learnify:
• Users → Join Call / Send Message / Share Screen → Learnify
• Learnify → Video & Audio Data / Chat Data → Users
• Learnify ↔ Signaling Server → Connection Signals

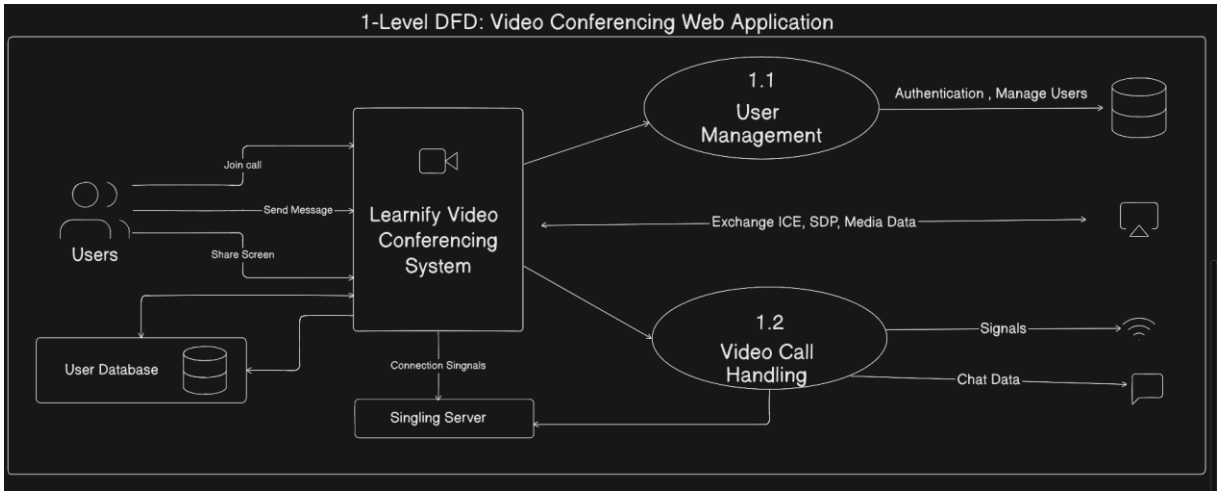


2. Level 1 DFD

The Level 1 DFD breaks the Learnify System into major sub-processes, such as:

• User Management
• Video Call Handling
• Chat Processing
• Screen Sharing
• Connection Signaling

It shows the interaction between users, system processes, signaling server, and data stores like the User Database.



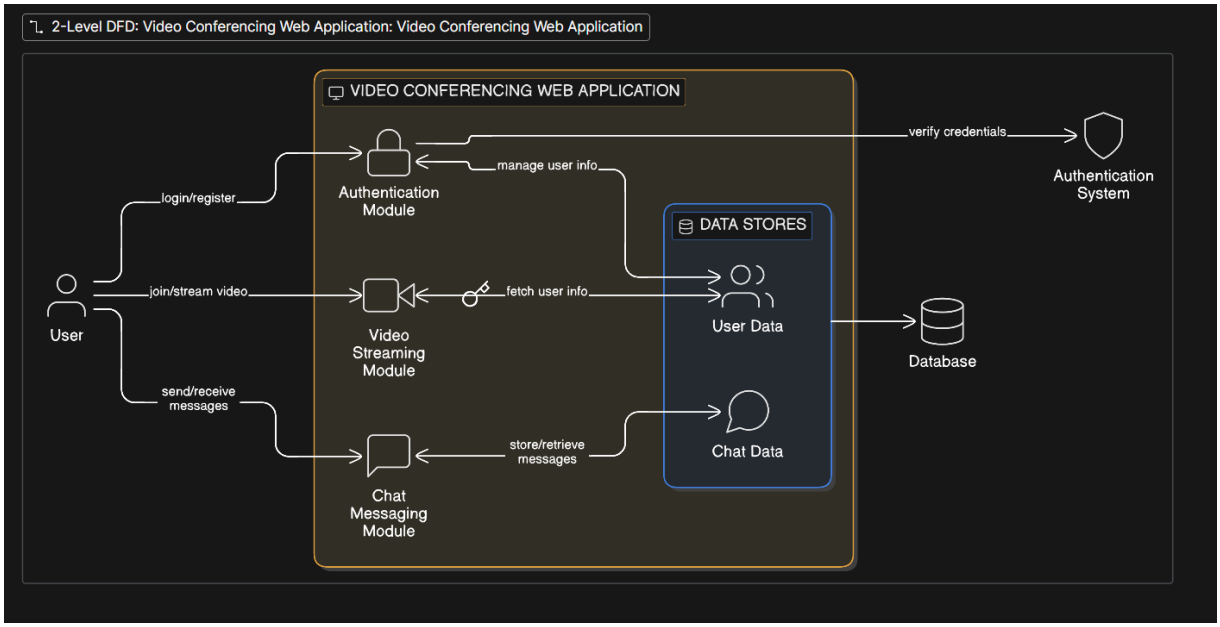
3. Level 2 DFD

The Level 2 DFD provides a detailed decomposition of major processes in the Learnify System.

Example sub-processes:

- Verify login/register credentials
- Manage user information
- Fetch user data
- Start/join video stream
- Send/receive chat messages
- Store/retrieve chat data

This level shows how authentication, video streaming, and chat modules interact with the database and handle data processing.



PRACTICAL NO-4

Objective: Design an ER diagram for with multiplicity

Work Done / Result:
Definition:
An Entity Relationship (ER) Diagram represents:

- Entities (tables)
- Attributes (fields)
- Relationships between entities
- Cardinality / Multiplicity (1:1, 1:M, M:N)

It is mainly used for database design.

ER Diagram for Learnify – Video Conferencing System

Entities & Attributes

User
• User_ID (PK)
• Name
• Username (Unique)
• Email
• Password (Hashed)
• Role (Student / Instructor / Admin)
• Token (Optional)

Meeting
• Meeting_ID (PK)
• Meeting_Code (Unique)
• Title
• Description
• Created_Date (Default: Current Timestamp)
• Status (Active / Ended)
• Host_ID (FK → User_ID)

Participant
• User_ID (PK, FK → User_ID)
• Meeting_ID (PK, FK → Meeting_ID)
• Joined_At
• Role_In_Meeting (Host / Participant)

Message
• Message_ID (PK)
• Content
• Sent_At (Timestamp)
• User_ID (FK → User_ID)
• Meeting_ID (FK → Meeting_ID)

ScreenShare
• ScreenShare_ID (PK)

• Started_At
• Ended_At
• User_ID (FK → User_ID)
• Meeting_ID (FK → Meeting_ID)

Admin
• Admin_ID (PK)
• Username
• Email

Relationships with Cardinality

Relationship	**Cardinality**
User creates Meeting	1 : M
User joins Meeting (via Participant)	M : M
Meeting has Participants	1 : M
Meeting has Messages	1 : M
User sends Message	1 : M
User shares Screen in Meeting	1 : M
Meeting has ScreenShare	1 : M
Admin manages Users/Meetings	1 : M